

DATA STRUCTURE



MIPS Assembly Expression Evaluation Implementation

“Using Stack Data Structure”

Computer Organization

Team Members

Group Members	B.N
Ahmed Bahy Youssef	2
Ahmed Hossam Mohamed Fekry	3
Ahmed Ragab Mahmoud	4
Ahmed Sobhy Refaay	5
Hassan Hussien Azmy	15
Abdelrahman Mahmoud Mohamed	34

Table of content

Introduction	Page. 2
Stack Data Structure	Page. 3
Implementation	Page. 5
Code Screenshots	Page. 16
Test Cases	Page. 41

Introduction

In computer science, expression parsing, and manipulation play a fundamental role in various applications ranging from compilers to calculators. One crucial transformation is converting an infix expression, where operators are placed between operands, to a postfix expression, also known as Reverse Polish Notation (RPN), where operators follow their operands.

This project focuses on implementing an infix to postfix conversion algorithm using stacks in MIPS assembly language. MIPS assembly, a RISC (Reduced Instruction Set Computing) architecture, provides a platform to delve into low-level programming concepts, including stack manipulation and character processing.

The primary goal of this project is to develop an efficient algorithm to parse infix expressions and generate equivalent postfix expressions while preserving operator precedence rules. This involves managing two stacks: one for operators and another for operands and processing each character of the infix expression accordingly.

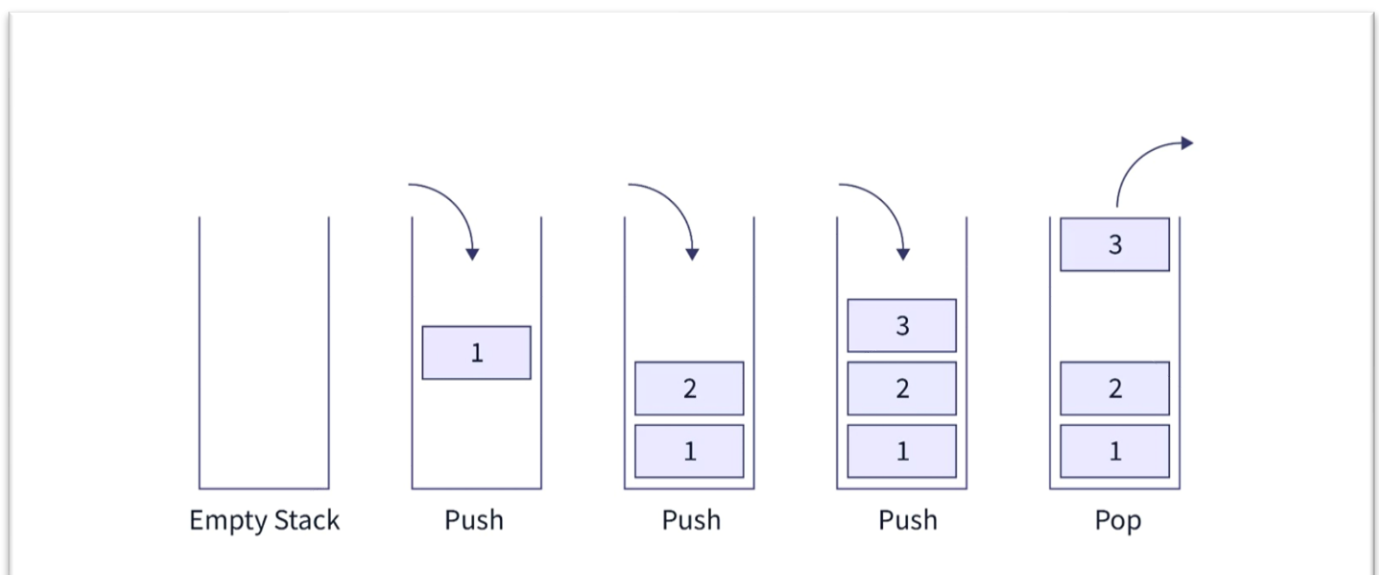
By embarking on this project, we aim to deepen our understanding of stack-based algorithms, character manipulation, and the MIPS assembly language. Through practical implementation and experimentation, we will gain insights into the intricacies of expression parsing and enhance our proficiency in low-level programming paradigms.

Stack Data Structure

A **Stack** is a linear data structure that follows a particular order in which the operations are performed. The order may be **LIFO (Last In First Out)** or **FILO(First In Last Out)**. **LIFO** implies that the element that is inserted last, comes out first and **FILO** implies that the element that is inserted first, comes out last.

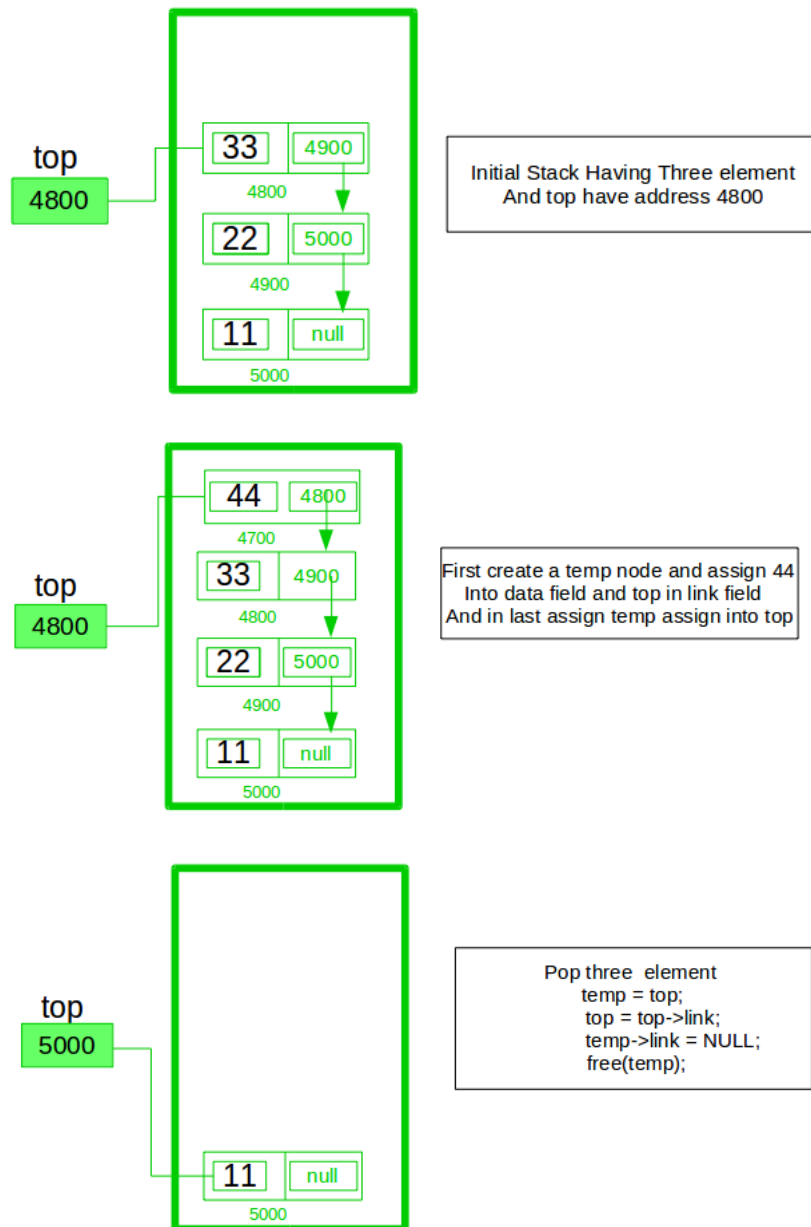
Key Operations on Stack Data Structures

- **Push**: Adds an element to the top of the stack.
- **Pop**: Removes the top element from the stack.
- **Top**: Returns the top element without removing it.
- **IsEmpty**: Checks if the stack is empty.
- **IsFull**: Checks if the stack is full.



Implement a stack using a singly linked list

To implement a stack using the singly linked list concept, all the singly linked operations should be performed based on Stack operations LIFO(last in first out), and with the help of that knowledge, we are going to implement a stack using a singly linked list.



In the stack Implementation, a stack contains a top pointer. which is the “head” of the stack were pushing and popping items happen at the head of the list. The first node has a null in the link field and second node-link has the first node address in the link field and so on and the last node address is in the “top” pointer.

The main advantage of using a linked list over arrays is that it is possible to implement a stack that can shrink or grow as much as needed. Using an array will put a restriction on the maximum capacity of the array which can lead to stack overflow. Here each new node will be dynamically allocated. so, overflow is not possible.

Implementation

Why postfix representation of the expression?

The compiler scans the expression either from left to right or from right to left.

Consider the expression: $a + b * c + d$

- The compiler first scans the expression to evaluate the expression $b * c$, then again scans the expression to add a to it.
- The result is then added to d after another scan.

The repeated scanning makes it very inefficient. Infix expressions are easily readable and solvable by humans whereas the computer cannot differentiate the operators and parenthesis easily so, it is better to convert the expression to postfix (or prefix) form before evaluation.

The corresponding expression in postfix form is $abc*+d+$. The postfix expressions can be evaluated easily using a stack.

Infix to Postfix Conversion

Expression = $A + B * C / D - F + A \wedge E$

Scanned Symbol	Stack	Output	Reason
A		A	Step 2
+	+	A	Step 3.1
B	+	AB	Step 2
*	+	AB	Step 3.1
C	+	ABC	Step 2
/	+/	ABC*	Step 3.2 / prec. is equal to * so not higher, so going to step 3.2
D	+/	ABC*D	Step 2
-	-	ABC*D/+	Step 3.2 / will be popped, added to o/p & then + popped & o/p. - will be pushed
F	-	ABC*D/+F	Step 2
+	+	ABC*D/+F-	Step 3.2 - will be popped, added to o/p and then + to stack
A	+	ABC*D/+F-A	Step 2
^	^	ABC*D/+F-A	Step 2
E	^	ABC*D/+F-AE	Step 2
(empty)		ABC*D/+F-AE^+	Step 8

How to convert an Infix to a Postfix expression?

To convert infix expression to postfix expression, use the [stack data structure](#). Scan the infix expression from left to right. Whenever we get an operand, add it to the postfix expression, and if we get an operator or parenthesis add it to the stack by maintaining their precedence.

Below are the steps to implement the above idea:

1. Scan the infix expression **from left to right**.
2. If the scanned character is an operand, put it in the postfix expression.
3. Otherwise, do the following
 - If the precedence and associativity of the scanned operator are greater than the precedence and associativity of the operator in the top stack [or the stack is empty or the stack contains a '('], then push it in the stack. ['^' operator is right associative and other operators like '+', '-', '*', and '/' are left-associative].
 - Check especially for a condition when the operator at the top of the stack and the scanned operator both are '^'. In this condition, the precedence of the scanned operator is higher due to its right associativity. So it will be pushed into the operator stack.
 - In all the other cases when the top of the operator stack is the same as the scanned operator, then pop the operator from the stack because of left associativity due to which the scanned operator has less precedence.
 - Else, Pop all the operators from the stack which are greater than or equal to in precedence than that of the scanned operator.
 - After doing that Push the scanned operator to the stack. (If you encounter parenthesis while popping then stop there and push the scanned operator in the stack.)
4. If the scanned character is a '(', push it to the stack.
5. If the scanned character is a ')', pop the stack and output it until a '(' is encountered, and discard both the parenthesis.
6. Repeat steps **2-5** until the infix expression is scanned.
7. Once the scanning is over, Pop the stack and add the operators in the postfix expression until it is not empty.
8. Finally, print the postfix expression.

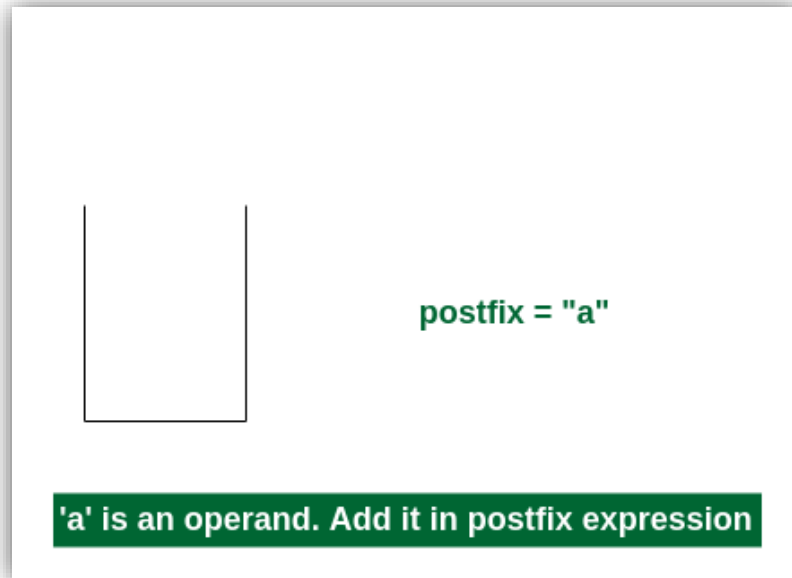
Illustration:

Follow the below illustration for a better understanding

Consider the infix expression $exp = "a+b*c+d"$

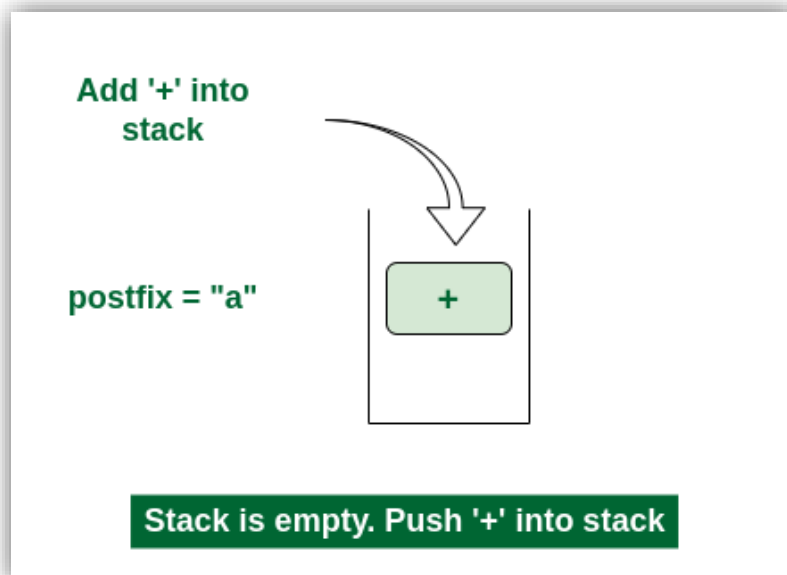
and the infix expression is scanned using the iterator i , which is initialized as $i = 0$.

1st Step: Here $i = 0$ and $exp[i] = 'a'$ i.e., an operand. So add this in the postfix expression. Therefore, postfix = "a".



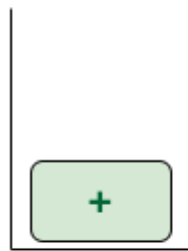
Add 'a' in the postfix

2nd Step: Here $i = 1$ and $exp[i] = '+'$ i.e., an operator. Push this into the stack. postfix = "a" and stack = {+}.



Push '+' in the stack.

3rd Step: Now $i = 2$ and $exp[i] = 'b'$ i.e., an operand. So add this in the postfix expression. postfix = "ab" and stack = {+}.



postfix = "ab"

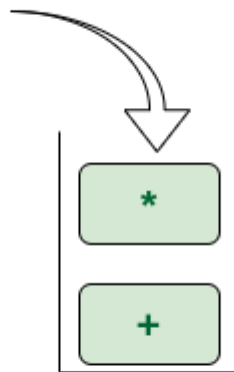
'b' is an operand. Add it in postfix expression

Add 'b' in the postfix

4th Step: Now $i = 3$ and $\text{exp}[i] = '*'$ i.e., an operator. Push this into the stack. **postfix = "ab"** and **stack = {+, *}**.

Add '*' into stack

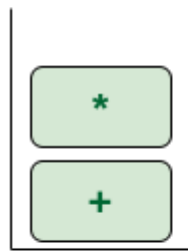
postfix = "ab"



* has higher precedence. Push it into stack

Push '' in the stack.*

5th Step: Now $i = 4$ and $\text{exp}[i] = 'c'$ i.e., an operand. Add this in the postfix expression. **postfix = "abc"** and **stack = {+, *}**.



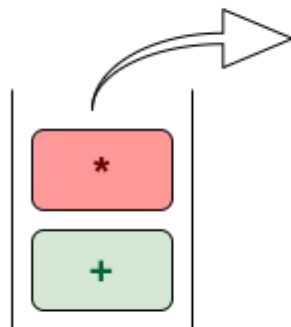
postfix = "abc"

'c' is an operand. Add it in postfix expression

Add 'c' in the postfix.

6th Step: Now $i = 5$ and $\text{exp}[i] = '+'$ i.e., an operator. The topmost element of the stack has higher precedence. So pop until the stack becomes empty or the top element has less precedence. '*' is popped and added in postfix. So postfix = "abc*" and stack = {+}.

Add to postfix

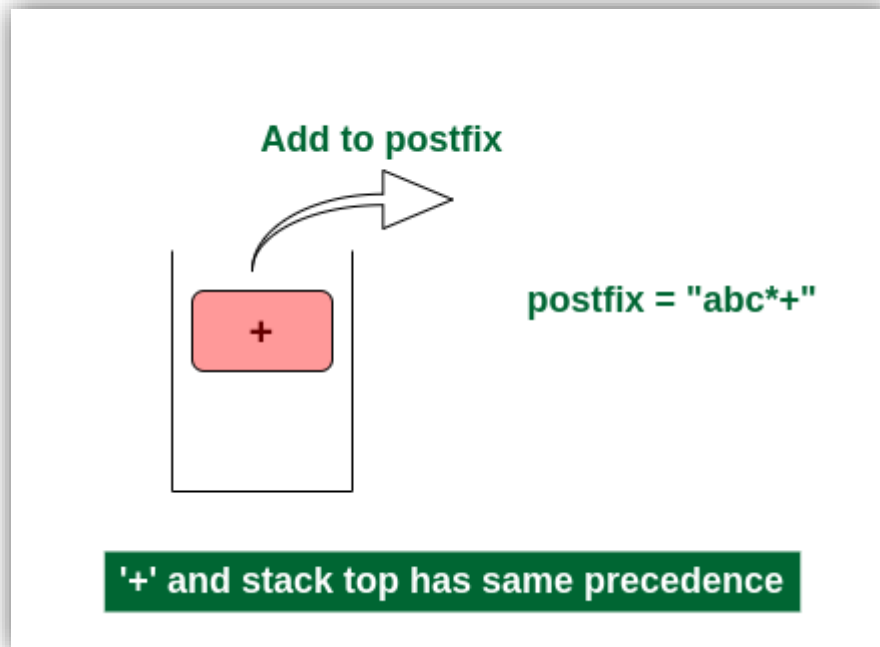


postfix = "abc*"

stack top has higher precedence than +

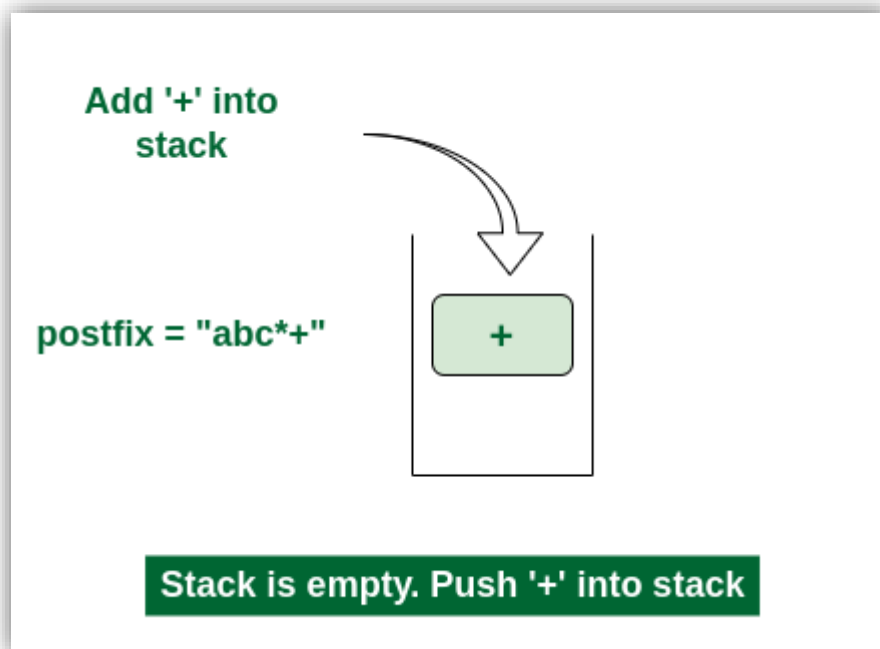
Pop '' and add in postfix.*

Now top element is '+' that also doesn't have less precedence. Pop it. postfix = "abc*+".



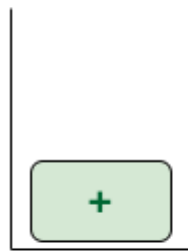
Pop '+' and add it in postfix.

Now stack is empty. So push '+' in the stack. stack = {+}.



Push '+' in the stack.

7th Step: Now $i = 6$ and $\text{exp}[i] = 'd'$ i.e., an operand. Add this in the postfix expression. postfix = "abc*+d".



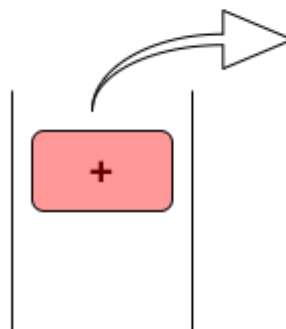
postfix = "abc*+d"

'd' is an operand. Add it in postfix expression

Add 'd' in the postfix.

Final Step: Now no element is left. So, empty the stack and add it in the postfix expression. postfix = "abc+d+".*

Add to postfix



postfix = "abc*+d+"

Nothing left. So pop all operators

Pop '+' and add it in postfix.

2 Evaluation

. Using postfix notation, operators are placed after their operands.

. Use a stack to store operands and intermediate results.

. Iterate through each symbol in the expression:

>> If it's an operand, push it onto the stack.

>> If it's an operator, pop 2 operands from the stack, apply the operator, and push the result back onto the stack.

After processing all symbols, the final result will be the only element remaining on the stack.

3 Validation & Standard Form

What about $4-3/-3*-(5+3)(3/-2)$?

So, We need convert into **Standard Form**

Handeled Cases

- If "-" follows an operator, adds "(0-1)*" between them
- If "-" follows "(" or "*", adds "(0-1)*" between them
- If "-" follows "/" , adds "(0-1)*" between them
- Adds "*" before "(" if it follows a digit or another ")"
- Skips adding "+" if it follows another operator or a "(" directly or came in the front of expression

INPUT FORM

$+2 + 2$

$2 - -5$

$4 + 12/-3$

$5+5/-3*-3$

$4(-3+-3)$

$(4+3)(5-3)$

STANDARD FORM

$2 + 2$

$2 - (0-1)*5$

$4 + 12/(0-1)/3$

$5+5/(0-1)/3*(0-1)*3$

$4*((0-1)*3-3)$

$(4+3)*(5-3)$

Valid Expression Rules

- Be Balance.
- Must start with open parentheses or with digit.
- Operators must have operands on both sides.
- No mismatched parentheses or operands

Balance

- Ensures that for every opening parenthesis, there is a corresponding closing parenthesis, and vice versa.

-Examples of unbalanced expressions:

)() ()()(
(())

-Examples of balanced expressions

() (((()))) ()(())

Code Screenshots

1. Stack operations & Creation: -

Push

Mips

```
1078 #a0 = value to be push
1079 push:
1080 # use $t5 , $t0 , $t1
1081 addi $sp , $sp , -12
1082 sw $t0 , 0 ($sp)
1083 sw $t1 , 4 ($sp)
1084 sw $t2 , 8 ($sp)
1085 add $t2,$zero,$a0 # assign the value to be pushed in $t2
1086
1087 # System call for memory allocation
1088 li $v0, 9
1089 # Allocate memory for 2 words (node structure)
1090 li $a0, 8
1091 # Allocate memory and store address in $v0
1092 syscall
1093
1094 #move $a0 , $t2
1095 # Store the data value in the node's 'value' field
1096 sw $t2, 0($v0)
1097
1098 # Link the new node to the current head of the linked list
1099 # Load the current head pointer
1100 lw $t0, head
1101 # Store the address of the current head in 'next' field of the new node
1102 sw $t0, 4($v0)
1103 # Update the head pointer to point to the new node
1104 sw $v0, head
1105
1106 # Increment the stack size
1107 # Load the current stack size
1108 lw $t1, stack_size
1109 # Increment the stack size
1110 addi $t1, $t1, 1
1111 # Store the updated stack size
1112 sw $t1, stack_size
1113
1114 lw $t0 , 0 ($sp)
1115 lw $t1 , 4 ($sp)
1116 lw $t2 , 8 ($sp)
1117 addi $sp , $sp, 12
1118 jr $ra
```

C++

```
31
32 void push(int data)
33 {
34
35     // Create new node temp and allocate memory in heap
36     Node* temp = new Node(data);
37
38     // Initialize data into temp data field
39     temp->data = data;
40
41     // Put top pointer reference into temp link
42     temp->link = top;
43
44     // Make temp as top of Stack
45     top = temp;
46     SIZE++;
47 }
```

```

1120 # Pop operation to remove a node from the stack
1121 pop:
1122     #use $t0, $t2 and save $ra
1123     addi $sp, $sp, -12
1124     sw $ra, 0($sp)
1125     sw $t0, 4($sp)
1126     sw $t2, 8($sp)
1127
1128     #lw $t0, stack_size
1129     lw $t0, head
1130     # Check if the stack is empty (head is null)
1131     beq $t0, $zero, pop_empty
1132
1133     # Update the head pointer to point to the next node
1134     # Load the address of the next node
1135     lw $t0, 4($t0)
1136     # Update the head pointer
1137     sw $t0, head
1138
1139     # Decrement the stack size
1140     # Load the current stack size
1141     lw $t2, stack_size
1142     # Decrement the stack size
1143     addi $t2, $t2, -1
1144     # Store the updated stack size
1145     sw $t2, stack_size
1146
1147     j exitPop
1148 pop_empty:
1149     # Stack is empty, handle error or return safely
1150     la $a0, emptyStack
1151     jal printString
1152 exitPop:
1153     lw $ra, 0($sp)
1154     lw $t0, 4($sp)
1155     lw $t2, 8($sp)
1156     addi $sp, $sp, 12
1157     jr $ra # return to the caller (evaluation function )

```

```

70 void pop()
71 {
72     Node* temp;
73
74     // Check for stack underflow
75     if (top == NULL) {
76         cout << "\nStack Underflow" << endl;
77         exit(1);
78     }
79     else {
80
81         // Assign top to temp
82         temp = top;
83
84         // Assign second node to top
85         top = top->link;
86         SIZE--;
87
88         // This will automatically destroy
89         // the link between first node and second node
90
91         // Release memory of top node
92         // i.e delete the node
93         free(temp);
94     }
95 }

```

Print

Mips

```
1216 # function to print stack elements
1217 print:
1218     addi $sp, $sp, -8
1219     sw $ra, 0($sp)
1220     sw $t0, 4($sp)
1221
1222     lw $t0, head           # the top pointer
1223     bne $t0, $zero, loopPrint  # check if it's empty
1224
1225     la $a0, emptyStack      # message
1226     jal printString
1227
1228     j exitLoopPrint
1229 loopPrint:
1230     # check if the pointer reached null
1231     beq $t0, $zero, exitLoopPrint
1232     # printing the current value
1233     lw $a0, 0($t0)
1234     jal printInteger
1235     jal printNewLine
1236     # moving to the next node
1237     lw $t0, 4($t0)
1238     j loopPrint
1239 exitLoopPrint:
1240     lw $ra, 0($sp)
1241     lw $t0, 4($sp)
1242     addi $sp, $sp, 8
1243     jr $ra
```

C++

```
88
89 // Function to print all the
90 // elements of the stack
91 void print()
92 {
93     Node* temp;
94
95     // Check for stack underflow
96     if (top == NULL) {
97         cout << "\nStack Underflow";
98         exit(1);
99     }
100     else {
101         temp = top;
102         while (temp != NULL) {
103
104             // Print node data
105             cout << temp->data;
106
107             // Assign temp link to temp
108             temp = temp->link;
109             if (temp != NULL)
110                 cout << " -> ";
111         }
112     }
113 }
```

Top

Mips

```
1165 # Top operation to return the value at the top of the stack
1166 top:
1167     # lw $t0, stack_size
1168     # use $t0 so save it to the Memory
1169     addi $sp, $sp, -4
1170     sw $t0, 0($sp)
1171
1172     lw $t0, head
1173     # Check if the stack is empty (head is null)
1174     beq $t0, $zero, top_empty
1175     # Load the value stored in the top node
1176     lw $v0, 0($t0)
1177     # Return to the caller
1178     j exit_top
1179
1180 top_empty:
1181     # Stack is empty, handle error or return safely
1182     # Return -1 (or set to an appropriate error value)
1183     li $v0, -1
1184
1185 exit_top:
1186     lw $t0, 0($sp)
1187     addi $sp, $sp, 4
1188     jr $ra
```

C

```
51 // Utility function to return top element in a stack
52 int TOP()
53 {
54     // If stack is not empty, return the top element
55     if (!isEmpty())
56         return top->data;
57     else
58         exit(1);
59 }
```

Size

Mips



```
1159 # Get the size of the stack
1160 size:
1161     # Load the stack size
1162     lw $v0, stack_size
1163     jr $ra
```

C++



```
123 int size()
124 {
125     return SIZE ;
126 }
```

FREE

Mips



```
1198 # function to delete all element to the stack
1199 free:
1200     # it use $t0 so save it to the memory
1201     addi $sp , $sp , -4
1202     sw $t0 , 0 ($sp )
1203     #load the head to $t0
1204     lw $t0, head
1205     #loop untill $t0 = 0 (first head )
1206     loopEmpty:
1207         beq $t0, $zero, exitLoopEmpty
1208         lw $t0, 4($t0)
1209         j loopEmpty
1210     exitLoopEmpty:
1211         sw $t0, head
1212         lw $t0 , 0 ($sp)
1213         addi $sp , $sp ,4
1214         jr $ra
```

IsEmpty

Mips



```
1190 isEmpty:
1191     # Load the head pointer
1192     lw $v0, head
1193     # Set $v0 to 1 if head is null
1194     #otherwise 0
1195     seq $v0, $v0, $zero
1196     jr $ra
```

C++



```
42 // Utility function to check if
43 // the stack is empty or not
44 bool isEmpty()
45 {
46     // If top is NULL it means that
47     // there are no elements are in stack
48     return top == NULL;
49 }
```

2. Infix To Postfix Algorithm: -

Infix2Postfix

C++

```
19
20 vector<string> infix_to_postfix(string exp)
21 {
22     stack<char> stk;
23     vector<string> output;
24     int temp1 = 0;
25     for (int i = 0; i < exp.length(); i++)
26     {
27         if (exp[i] == ' ')
28             continue;
29         if (isdigit(exp[i]))
30         {
31             while (isdigit(exp[i]))
32             {
33                 int c = exp[i] - '0';
34                 temp1 = temp1 * 10 + c;
35                 i++;
36             }
37             i--;
38             string val = to_string(temp1);
39             output.push_back(val);
40             temp1 = 0;
41         }
42         else if (exp[i] == '(')
43             stk.push('(');
44         else if (exp[i] == ')')
45         {
46             while (stk.top() != '(')
47             {
48                 string xx = string(1, stk.top());
49                 output.push_back(xx);
50                 stk.pop();
51             }
52             stk.pop();
53         }
54         else
55         {
56             while (!stk.empty() && Priority(exp[i]) <= Priority(stk.top()))
57             {
58                 if (exp[i] == '^' && stk.top() == '^')
59                     break;
60                 string xx = string(1, stk.top());
61                 output.push_back(xx);
62                 stk.pop();
63             }
64             stk.push(exp[i]);
65         }
66     }
67     while (!stk.empty())
68     {
69         string xx = string(1, stk.top());
70         output.push_back(xx);
71         stk.pop();
72     }
73     return output;
74 }
75
```

pushToString



```
718 # void pushToString(char c);
719 pushToString:
720     # index where we will push the char
721     lw $t0, tempStringSize
722     # tempString[size] = c
723     sb $a1, tempString($t0)
724     # size ++
725     addi $t0, $t0, 1
726     # updating the size of the tempString
727     sw $t0, tempStringSize
728     jr $ra
```

PushString2Array

Mips

```
730 # void pushStringToArray(string str);
731 pushStringToArray:
732     addi $sp, $sp, -4
733     sw $ra, 0($sp)
734
735     # last index of the tempString
736     lw $t0, tempStringSize
737     # pushing a null terminator
738     sb $zero, tempString($t0)
739
740     # Allocating memory
741     # allocating a memory whose address will be stored in $v0
742     li $v0, 9
743     # size of the memory allocated (10 bytes)
744     li $a0, 10
745     syscall
746     # $t2 = address for the memory allocated
747     move $t9, $v0
748
749     # copyString 1st's argument
750     la $a1, tempString
751     # copyString 2nd's argument
752     move $a2, $t9
753     # copyString(str1, str2)
754     jal copyString
```

```
756 # $t3 = array_size
757 lw $t3, array_size
758 # offset = $t3 * 4
759 sll $t3, $t3, 2
760 # pushing the allocated memory address in the array
761 sw $t9, string_array($t3)
762
763 # $t3 = array_size
764 lw $t3, array_size
765 # incrementing the size
766 addi $t3, $t3, 1
767 # updating the size
768 sw $t3, array_size
769
770 # $t0 = 0
771 li $t0, 0
772 # pointer of the tempString to index 0
773 sw $t0, tempStringSize
774
775 lw $ra, 0($sp)
776 addi $sp, $sp, 4
777 jr $ra
```

toString

```
651 toString: # String toString(int num)
652     addi $sp, $sp, -4
653     sw $ra, 0($sp)
654
655     li $t1, 10 # base
656     move $t2, $a0 # num
657     li $t3, 0 # i
658     convert_loop:
659         # num / 10
660         div $t2, $t1
661         # $t9 = num % 10
662         mfhi $t9
663         # $t9 = char(num)
664         addi $t9, $t9, 48
665         # storing the char in tempString
666         sb $t9, tempString($t3)
667         # get the quotient
668         mflo $t2
669         # increment i
670         addi $t3, $t3, 1
671         # repeat until quotient is zero
672         bnez $t2, convert_loop
673         j reverse
674
675     reverse:
676         li $t1, 0
677         # store the size of the tempString
678         sw $t3, tempStringSize
679         # the last index of the string
680         addi $t3, $t3, -1
```

```
682 reverse_loop:
683     # Exit loop if start index >= end index
684     bge $t1, $t3, reverse_done
685     # Load character at start index
686     lb $t2, tempString($t1)
687     # Load character at end index
688     lb $t8, tempString($t3)
689     # Store character from end index at start index
690     sb $t8, tempString($t1)
691     # Store character from start index at end index
692     sb $t2, tempString($t3)
693     # Move start index forward
694     # Move end index backward
695     addi $t1, $t1, 1
696     addi $t3, $t3, -1
697     # loop
698     j reverse_loop
699
700 reverse_done:
701     lw $ra, 0($sp)
702     addi $sp, $sp, 4
703     jr $ra
```


Mips

Periority

```

800 periority:
801     addi $sp, $sp, -4
802     sw $t5, 0($sp)
803
804     # ASCII for '+'
805     li $t1, '+'
806     # ASCII for '-'
807     li $t8, '-'
808     # ASCII for '*'
809     li $t9, '*'
810     # ASCII for '/'
811     li $t0, '/'
812     # ASCII for '^'
813     li $t5, '^'
814
815     # Check the input character and return priority
816     beq $a0, $t1, plus_minus_label
817     beq $a0, $t8, plus_minus_label
818     beq $a0, $t9, multiply_division_label
819     beq $a0, $t0, multiply_division_label
820     beq $a0, $t5, exponential_label
821
822     li $v0, 0
823     j returnPriority

```

```

825 plus_minus_label:
826     # Return 1 for '+' or '-'
827     li $v0, 1
828     j returnPriority
829
830 multiply_division_label:
831     # Return 2 for '*' or '/'
832     li $v0, 2
833     j returnPriority
834
835 exponential_label:
836     # Return 3 for '^'
837     li $v0, 3
838     j returnPriority
839
840 returnPriority:
841     lw $t5, 0($sp)
842     addi $sp, $sp, 4
843     jr $ra

```

C++

```

8  int Priority(char c)
9  {
10     if (c == '-' || c == '+')
11         return 1;
12     else if (c == '*' || c == '/')
13         return 2;
14     else if (c == '^')
15         return 3;
16     else
17         return 0;
18 }
19

```

IsDigit

Mips

```

705 isDigit: # bool isDigit(char c);
706     # Set the default value to 0
707     li $v0, 0
708     # if(exp[i] < '0') return 0
709     blt $a1, '0', end_isDigit
710     # if(exp[i] > '9') return 0
711     bgt $a1, '9', end_isDigit
712     # if(exp[i] >= '0' && exp[i] <= '9') return 1
713     li $v0, 1
714
715 end_isDigit:
716     jr $ra

```

C++

```

134 isdigit(char ch)
135 {
136     return ch >='0' && ch <='9' ;
137 }

```

```

459 Infix_to_postfix:
460     addi $sp, $sp, -4
461     sw $ra, 0($sp)
462
463     # temp --> to split the expression
464     li $t2, 0
465     # counter
466     li $t4, 0
467     stringLoop:
468         # $t5 = expression[i]
469         lb $t5, expression($t4)
470         # if(expression[i] == '\n') break
471         beq $t5, '\n', exitStringLoop
472         # if(null-terminated) break
473         beq $t5, $zero, exitStringLoop
474         # if(expression[i] == ' ') continue
475         beq $t5, ' ', increment
476
477         # isDigit's argument
478         move $a1, $t5
479         # $v0 = isDigit(expression[i])
480         jal isDigit
481         # if(isOperator(expression[i])) jump
482         beq $v0, $zero, operator

```

```

483 innerLoop:
484     # if(expression[i] == ' ') continue
485     beq $t5, ' ', increment
486     # $t3 = int(expression[i])
487     addi $t3, $t5, -48
488     # temp = temp * 10
489     mul $t2, $t2, 10
490     # temp = temp * 10 + int(exp[i])
491     add $t2, $t2, $t3
492     # i ++
493     addi $t4, $t4, 1
494
495     # loading the next char
496     lb $t5, expression($t4)
497     # if(expression[i] == '\n') break
498     beq $t5, '\n', exitStringLoop
499     # if(null terminated) break
500     beq $t5, $zero, exitStringLoop
501
502     # isDigit's argument
503     move $a1, $t5
504     # $v0 = isDigit(expression[i])
505     jal isDigit
506     # if(!isDigit(exp[i])) break
507     bne $v0, $zero, innerLoop

```

```

508 exitInnerLoop:
509     # i --
510     addi $t4, $t4, -1
511     # toString's argument
512     move $a0, $t2
513     # toString(temp)
514     jal toString
515     # pushStringToArray()
516     jal pushStringToArray
517     # temp = 0
518     li $t2, 0
519     # i++ and loop
520     j increment
521 operator:
522     beq $t5, '(', leftParenthesis
523     beq $t5, ')', rightParenthesis
524     bge $t5, 42, else
525
526     # if(expression[i] == '(') push
527     leftParenthesis:
528         # argument
529         move $a0, $t5
530         # stackPush(exp[i])
531         jal push
532         # i++ and loop
533         j increment

```

```

535 rightParenthesis:
536     parenthesisLoop:
537         # $v0 = stackTop()
538         jal top
539         # $t6 = $v0
540         move $t6, $v0
541         # loop until '(' is found
542         beq $t6, '(', exitParenthesisLoop
543
544         # pushToString's argument
545         move $a1, $t6
546         # pushToString(stackTop())
547         jal pushToString
548         # push the operator to the array
549         jal pushStringToArray
550         # stackPop()
551         jal pop
552         # loop
553         j parenthesisLoop
554 exitParenthesisLoop:
555     # stackPop()
556     jal pop
557     # i++ and loop
558     j increment

```

```

559 else:
560     elseLoop:
561         # $v0 = stackTop()
562         jal top
563         # $t6 = $v0
564         move $t6, $v0
565         # exit if stack is empty
566         beq $t6, -1, exitElseLoop
567
568         # if(stackTop() == '^') t0 = 1
569         seq $t0, $t6, '^'
570         # if(exp[i] == '^') t1 = 1
571         seq $t1, $t5, '^'
572         # $t1 = $t0 & $t1
573         and $t1, $t1, $t0
574         # if("^^") push(exp[i])
575         beq $t1, 1, exitElseLoop
576
577         # periority's argument
578         move $a0, $t5
579         # $v0 = periority(exp[i])
580         jal periority
581         # $t7 = returned value
582         move $t7, $v0
583
584         # periority's argument
585         move $a0, $t6
586         # $v0 = periority(stackTop())
587         jal periority
588         # pushing the largest priority
589         bgt $t7, $v0, exitElseLoop

```

```

591         # pushToString's argument
592         move $a1, $t6
593         # pushToString(stackTop())
594         jal pushToString
595         # push the operator to the array
596         jal pushStringToArray
597         # stackPop()
598         jal pop
599         # loop
600         j elseLoop
601     exitElseLoop:
602         # stackPush's argument
603         move $a0, $t5
604         # stackPush(exp[i])
605         jal push
606     increment:
607         # i++
608         addi $t4, $t4, 1
609         # loop
610         j stringLoop
611 exitStringLoop:
612     # i-- --> expression.size
613     addi $t4, $t4, -1
614     # $t1 = expression[size - 1]
615     lb $t1, expression($t4)
616     # if($t1 == ')') empty the stack
617     beq $t1, ')', stackNotEmptyLoop
618     # toString's argument
619     move $a0, $t2
620     # toString(exp[size - 1])
621     jal toString
622     # pushStringToArray()
623     jal pushStringToArray

```



```
625  stackNotEmptyLoop:
626      # $v0 = stackTop()
627      jal top
628      # $t6 = $v0
629      move $t6, $v0
630      # if(stackEmpty()) exit
631      beq $t6, -1, exitStackNotEmptyLoop
632
633      # pushToString's argument
634      move $a1, $t6
635      # pushToString(stackTop())
636      jal pushToString
637      # pushStringToArray()
638      jal pushStringToArray
639      # stackPop()
640      jal pop
641      # loop
642      j stackNotEmptyLoop
643
644  exitStackNotEmptyLoop:
645
646      lw $ra, 0($sp)
647      addi $sp, $sp, 4
648      jr $ra
```

3. Expression Evaluation: -

C++

```
128
129 int Evaluation(vector<string>v)
130 {
131
132     stack <int> st ;
133     for (int i =0 ;i<v.size() ;i++)
134     {
135         if (is_operator(v[i][0]))
136         {
137             int b = st.top() ;
138             st.pop();
139
140             int a = st.top();
141             st.pop();
142             double val = make_operation(a, b, v[i][0]);
143             st.push(val);
144         }
145         else
146         {
147             int val = string_to_int(v[i]);
148             st.push(val) ;
149         }
150     }
151     return st.top();
152 }
153 }
```

is_operator

Mips

```
1042 isOperator:
1043     addi $sp, $sp, -24
1044     sw $t0, 0($sp)
1045     sw $t1, 4($sp)
1046     sw $t2, 8($sp)
1047     sw $t3, 12($sp)
1048     sw $t4, 16($sp)
1049     sw $t5, 20($sp)
1050
1051     move $v0, $zero
1052     move $t0, $a0
1053     li $t1, '+'
1054     li $t2, '-'
1055     li $t3, '*'
1056     li $t4, '/'
1057     li $t5, '^'
1058
1059     beq $t0, $t1, set
1060     beq $t0, $t2, set
1061     beq $t0, $t3, set
1062     beq $t0, $t4, set
1063     beq $t0, $t5, set
1064     j exit
1065     #exit label change boolean value
1066     set:
1067         addi $v0, $zero, 1
1068     exit:
1069         lw $t0, 0($sp)
1070         lw $t1, 4($sp)
1071         lw $t2, 8($sp)
1072         lw $t3, 12($sp)
1073         lw $t4, 16($sp)
1074         lw $t5, 20($sp)
1075         addi $sp, $sp, 24
1076         jr $ra
```

C++

```
81 bool is_operator(char c)
82 {
83     return (c == '+') || (c == '-') || (c == '*') || (c == '/') || (c == '^');
84 }
```

MakeOperation

C++

```
104 int make_operation(int a, int b, char op)
105 {
106     int val = 0;
107     switch (op)
108     {
109         case '+':
110             val = a+b;
111             break;
112         case '-':
113             val = a-b;
114             break ;
115         case '/':
116             val = a/b;
117             break;
118         case '*':
119             val = a*b;
120             break;
121
122         case '^':
123             val = power(a,b) ;
124         default:
125             break;
126     }
127     return val ;
128 }
129 }
```

Power

Mips

```
1013 power:
1014     #arguments: base ----> a, exponenet ----> a1      2,3
1015     # use $t0 , $t1 , $t2
1016     addi $sp , $sp, -12
1017     sw $t0, 0($sp)
1018     sw $t1, 4($sp)
1019     sw $t2, 8($sp)
1020     # result t1 = 1
1021     addi $t1, $0, 1
1022     # index = 0
1023     add $t0, $0, $0
1024     loop1:
1025         slt $t2, $t0, $a1
1026         # if index > exponenet ----> exit loop
1027         beq $t2, $0, exit_loop1
1028         # operation
1029         mul $t1, $t1, $a0
1030         # incrementing index
1031         addi $t0, $t0, 1
1032         j loop1
1033     exit_loop1:
1034         # result in V0
1035         add $v0, $t1, $0
1036         lw $t0 , 0($sp)
1037         lw $t1 , 4 ($sp)
1038         lw $t2 , 8 ($sp)
1039         addi $sp , $sp , 12
1040         jr $ra
```

C++

```
93  int power(int a ,int b )
94  {
95      // return a ^b
96      int value =1 ;
97      for (int i =0; i<b ;i++)
98      {
99          value *=a ;
100      }
101      return value;
102  }
```

Mips

```
952 makeoperation:
953     #arguments ----> operand1 a0, operand a1 and operation a2
954
955     #use $t0 so save it on the memory
956     addi $sp , $sp , -4
957     sw $t0 , 0($sp)
958     #checking if the operation is +
959     li $t0, '+'
960     beq $a2, $t0, plus_exit
961
962     #checking if the operation is -
963     li $t0, '-'
964     beq $a2, $t0, minus_exit
965
966     #checking if the operation is *
967     li $t0, '*'
968     beq $a2, $t0, mult_exit
969
970     #checking if the operation is /
971     li $t0, '/'
972     beq $a2, $t0, divison_exit
973
974     #checking if the operation is ^
975     li $t0, '^'
976     beq $a2, $t0, exponent_exit
977
978     # if the input is invalid
979     # flag ig operation is invalid
980     add $t1, $a0, $0
981     j exiteMakeoperation
```

MakeOperation

```
983 plus_exit:
984     add $v0, $a0, $a1
985     j exiteMakeoperation
986
987 minus_exit:
988     sub $v0, $a0, $a1
989     j exiteMakeoperation
990
991 mult_exit:
992     mul $v0, $a0, $a1
993     j exiteMakeoperation
994
995 divison_exit:
996     div $v0, $a0, $a1
997     j exiteMakeoperation
998
999 exponent_exit:
1000     addi $sp, $sp, -4
1001     sw $ra, 0($sp)
1002     # calling power function
1003     jal power
1004     lw $ra, 0($sp)
1005     addi $sp, $sp, 4
1006
1007     j exiteMakeoperation
1008 exiteMakeoperation:
1009     lw $t0 , 0($sp)
1010     addi $sp , $sp , 4
1011     jr $ra
```


Mips

string_to_int

```

912 strToInt:
913     # use $t0, $t1, $t2, $t3 so save them in memory
914     addi $sp, $sp, -16
915     sw $t0, 0($sp)
916     sw $t1, 4($sp)
917     sw $t2, 8($sp)
918     sw $t3, 12($sp)
919
920     # Loop counter i
921     li $t0, 0
922     # Initialize ans to 0
923     li $t1, 0
924     strToInt_loop:
925     # Load the byte (character) from the string into $t2
926     lb $t2, 0($a0)
927     # If the byte is 0 (null terminator), exit loop
928     beqz $t2, strToInt_done
929     # ASCII value of '0'
930     li $t3, 48
931     sub $t2, $t2, $t3
932     # Multiply ans by 10
933     mul $t1, $t1, 10
934     # Add the converted character to ans
935     add $t1, $t1, $t2
936     # Increment loop counter and pointer to next character
937     addi $t0, $t0, 1
938     addi $a0, $a0, 1
939     # Repeat the loop
940     j strToInt_loop
941
942 strToInt_done:
943     # Move the result to $v0
944     move $v0, $t1
945     lw $t0, 0($sp)
946     lw $t1, 4($sp)
947     lw $t2, 8($sp)
948     lw $t3, 12($sp)
949     addi $sp, $sp, 16
950     jr $ra

```

C++

```

84
85 long long string_to_int(string s)
86 {
87     long long val = 0 ;
88     for (char i:s)
89     {
90         val= val *10 + (i-'0') ;
91     }
92     return val;
93 }
94

```

Mips

Evaluation

C++

```

845 Evaluation:
846 #a0 address of the refrance array of strings
847 #loop for each string refrance in the array
848 #counter i
849 li $t7, 0
850 lw $t6, array_size
851 # address of refrance array
852 move $s2, $a0
853
854 # saving the return address in the stack
855 addi $sp, $sp, -4
856 sw $ra, 0($sp)
857 #move $t8, $ra
858 #ra is the position in the main function
859
860 loop:
861     beq $t7, $t6, exit_loop
862
863     # load the address of string
864     # refrance of string
865     lw $a0, 0($s2)
866     lb $a0, 0($a0)
867     beq $a0, $zero, increment2
868     jal isOperator
869     beq $v0, $0, Number

```

```

871 # get the top in v0
872 jal top
873 # b
874 move $t3, $v0
875 jal pop
876 # get the top in v0
877 jal top
878 # a
879 move $t4, $v0
880 jal pop
881 move $a0, $t4
882 move $t1, $t3
883 lw $t0, 0($t2)
884 lb $a2, 0($t0)
885 jal makeoperation
886 move $a0, $v0
887 jal push
888 increment2:
889     addi $t2, $t2, 4
890     addi $t7, $t7, 1
891     j loop
892 Number:
893     # refrance of string to a0
894     lw $a0, 0($s2)
895     jal strToInt
896     move $a0, $v0
897     jal push
898     addi $t2, $t2, 4
899     addi $t7, $t7, 1
900     j loop
901
902 exit_loop:
903     # result in $v0
904     jal top
905     lw $ra, 0($sp)
906     addi $t0, $t0, 4
907     jr $ra

```

```

128
129 int Evaluation(vector<string>v)
130 {
131
132     stack<int> st ;
133     for (int i =0 ; i<v.size() ;i++)
134     {
135         if (is_operator(v[i][0]))
136         {
137             int b = st.top() ;
138             st.pop();
139
140             int a = st.top();
141             st.pop();
142             double val = make_operation(a, b, v[i][0]);
143             st.push(val);
144         }
145         else
146         {
147             int val = string_to_int(v[i]);
148             st.push(val) ;
149         }
150     }
151     return st.top();
152 }
153

```

4. Validation & Standard Form: -

.data Section



```
1 .data
2 # Global variables
3     minusMul:                .asciiz "(0-1)*"
4     minusDiv:                .asciiz "(0-1)/"
```

toStandardInfix

C

```
178
179 string to_standard_infix(string s)
180 {
181     string res = "";
182     if (s[0] == '-')
183         res += "(0-1)*";
184     else if (s[0] != '+')
185         res += s[0];
186
187     for (int i = 1; i < s.size(); i++)
188     {
189         // 5 *+2 => 5 *2 remove unwanted +ve sign
190         if (s[i] == '+' && is_operator(s[i - 1]))
191             continue;
192         //      7(6) => 7*(6)          ||      (5)(8) => (5)*(8)
193         if ((s[i] == '(' && isdigit(s[i - 1])) || (s[i] == '(' && s[i - 1] == '('))
194             res += '*';
195         if (s[i] == '-' && is_operator(s[i - 1]))
196         {
197             if (s[i - 1] == '/')
198                 res += "(0-1)/"; // 5*-2 = 5*(0-1)*2
199             else
200                 res += "(0-1)*"; // 5/-2 = 5/(0-1)/2
201         }
202         else if (s[i] == '-' && s[i - 1] == '(')
203             res += "(0-1)*"; // (-2) = ((0-1)*2)
204         else
205             res += s[i];
206     }
207     return res;
208 }
```

```

268
269 #-----
270 # a1 --> the string to be copied ("0-1")
271 # copy loop function
272 Copy_additional_expression:
273
274     addi $sp , $sp , -16
275     sw $t0, 0($sp)
276     sw $t1, 4($sp)
277     sw $t2, 8($sp)
278     sw $t3, 12($sp)
279
280     la $t0 , res    # copy the refrance of string to be copied in it
281
282     move $t1 , $a1    # copy te refrance of string  to be copied ("0-1")
283
284     # s1 contain result size
285     add $t0 , $t0 , $s1    # start index to be stor in it
286
287 Copy_additional_expression_loop:
288
289     lb $t2, 0($t1)        # Load the first byte of $a1 (str1) into $t2
290     beq $t2, $zero, done_copy # if null so copy done and go out of the loop
291     sb $t2, 0($t0)        # Save the value in $t2 at the same byte in $s1 (str2)
292     addi $t0, $t0, 1        # Increment both memory locations by 1 byte
293     addi $t1, $t1, 1
294     addi $s1 , $s1,1    # increase the size of the result string
295     j Copy_additional_expression_loop
296
297 done_copy:
298     lw $t0 , 0($sp)
299     lw $t1 , 4 ($sp)
300     lw $t2 , 8 ($sp)
301     lw $t3 , 12 ($sp)
302     addi $sp , $sp , 16
303     jr $ra
304 #-----
305 # isvalid function
306 #a0 , refrance of string

```

Mips

```

150 toStandardInfix:
151     addi $sp,$sp,-4
152     sw $ra, 0($sp)
153
154     # counter i
155     li $t3, 0
156     # &filtered expression
157     la $s0, filteredExp
158     # standardExpression size
159     li $s1, 0
160     # &standardExpression
161     la $s2, standardExp
162
163 to_standard_infix_loop:
164     # t1 --> s[i]      t0 --> s[i-1]
165     # load from filteredExpression
166     lb $t1, 0($s0)
167     # if(null-terminator) break
168     beq $t1, $zero, end_to_standard_loop
169     # if(i != 0) jump
170     bne $zero,$t3, I_largThanZero
171     # if(s[0] == '-') push (0-1)*
172     beq $t1, '-', Edit_for_multiplication
173     # if(exp[0] == '+') continue
174     beq $t1, '+', next_iteration
175     # if(exp[0] != '-' && exp[0] != '+') push
176     j push_to_res

```

```

178 I_largThanZero:
179     # filteredExpression[i-1]
180     lb $t0, -1($s0)
181     # if(exp[i] != '+') jump
182     bne $t1, '+', continue
183
184     # if(exp[i] == '+')
185     # $v0 = isOperator(exp[i-1])
186     move $a0, $t0
187     jal isOperator
188     # if(exp[i] == '+' && isOperator(exp[i-1])) continue
189     beq $v0,$0, handlerightbracket
190     j next_iteration
191
192 handlerightbracket:
193     # if(exp[i] == '+' && exp[i-1] == '(') continue
194     beq $t0, '(', next_iteration
195     # else
196     j push_to_res
197
198     # if(exp[i] != '+')
199     continue:
200     # if(exp[i] != '(') jump
201     bne $t1, '(', continue2
202     # if )( --> push '*'
203     beq $t0, ')', add_asterisk

```

```

205 # isDigit(s[i-1])
206 move $a1, $t0
207 jal isDigit
208 # if(!isDigit(s[i-1])) jump
209 beq $v0,$0, continue2
210 # if A( --> push '*'
211 j add_asterisk
212
213 # if(exp[i] != '(' || !isDigit(exp[i-1]))
214 continue2:
215 # if(exp[i] != '-') push s[i]
216 bne $t1, '-', push_to_res
217
218 # if(exp[i] == '-')
219 # $v0 = isOperator(s[i-1])
220 move $a0, $t0
221 jal isOperator
222 # if(!isOperator(s[i-1])) jump
223 beq $v0,$0, not_operator
224
225 # if(isOperator(s[i-1]))
226 # if /- --> push (0-1)/
227 beq $t0, '/', Edit_for_division
228 # else --> push (0-1)"
229 j Edit_for_multiplication
230 not_operator:
231 # if (- --> push (0-1)*
232 beq $t0, '(', Edit_for_multiplication
233 # else
234 j push_to_res

```

```

236 Edit_for_multiplication:
237     # push (0-1)*
238     la $a1, minusMul
239     jal Copy_additional_expression
240     j next_iteration
241 Edit_for_division:
242     # push (0-1)/
243     la $a1, minusDiv
244     jal Copy_additional_expression
245     j next_iteration
246
247 add_asterisk:
248     # Load the ASCII code for '*'
249     li $t4, '*'
250     # next standardExp's idx
251     add $t8, $s2, $s1
252     # Storing an *
253     sb $t4, 0($t8)
254     # increase the size of the standardExp
255     addi $s1, $s1, 1
256     # pushing the '('
257     j push_to_res
258
259 push_to_res:
260     # next standardExp idx
261     add $t8, $s2, $s1
262     # store filteredExp[1] in standardExp
263     sb $t1, 0($t8)
264     # increase the size of the standardExp
265     addi $s1, $s1, 1

```

```

266     next_iteration:
267         # i ++
268         addi $t3, $t3, 1
269         addi $s0, $s0, 1
270         j to_standard_infix_loop
271 end_to_standard_loop:
272
273     add $t8, $s2, $s1
274     sb $zero, 0($t8)
275     lw $ra, 0($sp)
276     addi $sp, $sp, 4
277     jr $ra

```

IsValidExpression

C

```

195 bool isValidExpression(string s)
196 {
197     if(!Balance(s))return false;
198     int prev=-1; // 1 for digits , 2 for operators , 3 for '(', 4 for ')'
199     for(char c:s){
200         if(c=='(')
201             {
202                 if(prev==4 || prev==1)return false;
203                 prev=3;
204             }
205         else if(c==')')
206             {
207                 if(prev==1 || prev==3)return false;
208                 prev=4;
209             }
210         }else if(!isdigit(c))
211             {
212                 if(prev==4)return false;
213                 prev=1;
214             }else if(c=='-' || c=='+') || c=='*' || c=='/')
215             {
216                 if(prev==1 || prev==2 || prev==3)return false;
217                 prev=2;
218             }
219     }
220     if(prev==3 || prev==2)
221         return false;
222     return true;
223 }

```

Mips

```

195 isValidExpression:
196     addi $sp, $sp, -16
197     sw $ra, 0($sp)
198     sw $t0, 4($sp)
199     sw $t1, 8($sp)
200     sw $t2, 12($sp)
201     # startStandardExp
202     move $t2, $s0
203     # flag for the previous char
204     # 1-->digit, 2-->operator
205     # 3-->'(' , 4-->')'
206     li $t0, -1
207
208     InvalidLoop:
209     # load from standardExp
210     lb $t1, 0($t2)
211     # if('\'') break
212     beq $t1, '\n', end_loop
213     # if(not-terminator) break
214     beqz $t1, end_loop
215     # moving to next character
216     addi $t2, $t2, 1
217
218     # Check character
219     beq $t1, '(', check_open_bracket
220     beq $t1, ')', check_close_bracket
221
222     # call isDigit function $t0 return flag
223     move $t1, $t1
224     jal isDigit # check if character is a digit
225     bne $t0, $zero, checkDigit
226

```

```

227     # call is operator function
228     move $t0, $t1
229     jal isOperator
230     # if not a digit, check if it's an operator
231     bne $t0, $zero, check_operator
232
233     check_open_bracket:
234     # if previous was ')' or initial, return false
235     beq $t0, 4, unbalanced
236     # if previous was a digit, return false
237     beq $t0, 1, unbalanced
238     # set prev = 3 (for '(')
239     li $t0, 3
240     j InvalidLoop
241
242     check_close_bracket:
243     # if previous was initial, return false
244     beq $t0, -1, unbalanced
245     # if previous was '(', return false
246     beq $t0, 3, unbalanced
247     # if previous was operator , return false
248     beq $t0, 2, unbalanced
249     # set prev = 4 (for ')')
250     li $t0, 4
251     j InvalidLoop
252
253     checkDigit:
254     beq $t0, 4, unbalanced
255     li $t0, 1
256     j InvalidLoop

```

```

257     check_operator:
258     # if previous was initial, return false
259     beq $t0, -1, unbalanced
260     # if previous was an operator, return false
261     beq $t0, 2, unbalanced
262     # if previous was '(', return false
263     beq $t0, 3, unbalanced
264     # set prev = 2 (for operator)
265     li $t0, 2
266     j InvalidLoop
267
268     end_loop:
269     # if previous was '(', return false
270     beq $t0, 3, unbalanced
271     # if previous was an operator, return false
272     beq $t0, 2, unbalanced
273     # return 1 (true)
274     li $t0, 1
275     j end_expression
276
277     unbalanced:
278     # return 0 (false)
279     li $t0, 0
280
281     end_expression:
282     lw $ra, 0($sp)
283     lw $t0, 4($sp)
284     lw $t1, 8($sp)
285     lw $t2, 12($sp)
286     addi $sp, $sp, 16
287     jr $ra

```



```
195
196 bool isValidExpression(string s)
197 {
198     if(!Balance(s))return false;
199     int prev=-1;        // 1 for digits , 2 for operators , 3 for '(' , 4 for ')'
200     for(char c:s){
201         if(c=='(')
202         {
203             if(prev==4 || prev==1)return false;
204             prev=3;
205         }
206         else if(c==')')
207         {
208             if(prev==-1 || prev==3)return false;
209             prev=4;
210         }else if(isdigit(c))
211         {
212             if(prev==4)return false;
213             prev=1;
214         }else if(c=='-' || c=='+' || c=='*' || c=='/')
215         {
216             if(prev==-1 || prev==2 || prev==3)return false;
217             prev=2;
218         }
219     }
220     if(prev==3 || prev==2)
221         return false;
222     return true;
223 }
```

Mips



```
315 isValidExpression:
316     addi $sp, $sp, -16
317     sw $ra, 0($sp)
318     sw $t0, 4($sp)
319     sw $t1, 8($sp)
320     sw $t2, 12($sp)
321
322     # &standardExp
323     move $t2, $a0
324     # flag for the previous char
325     # 1-->digit, 2-->operator
326     # 3-->'(', 4-->')'
327     li $t0, -1
328
329     lInvalidLoop:
330     # load from standardExp
331     lb $t1, 0($t2)
332     # if('\n') break
333     beq $t1, '\n', end_loop
334     # if(null-terminator) break
335     beqz $t1, end_loop
336     # moving to next character
337     addi $t2, $t2, 1
338
339     # Check character
340     beq $t1, '(', check_open_bracket
341     beq $t1, ')', check_close_bracket
342
343     # call isDigit function    $v0 return flage
344     move $a1, $t1
345     jal isDigit              # check if character is a digit
346     bne $v0, $zero, checkDigit
```



```
348     # call is operator function
349     move $a0, $t1
350     jal isOperator
351     # if not a digit, check if it's an operator
352     bne $v0, $zero, check_operator
353
354     check_open_bracket:
355     # if previous was ')' or initial, return false
356     beq $t0, 4, unbalanced
357     # if previous was a digit, return false
358     beq $t0, 1, unbalanced
359     # set prev = 3 (for '(')
360     li $t0, 3
361     j lInvalidLoop
362
363     check_close_bracket:
364     # if previous was initial, return false
365     beq $t0, -1, unbalanced
366     # if previous was '(', return false
367     beq $t0, 3, unbalanced
368     # if previous was operator , return false
369     beq $t0, 2, unbalanced
370     # set prev = 4 (for ')')
371     li $t0, 4
372     j lInvalidLoop
373
374     checkDigit:
375     beq $t0, 4, unbalanced
376     li $t0, 1
377     j lInvalidLoop
```



```
378     check_operator:
379     # if previous was initial, return false
380     beq $t0, -1, unbalanced
381     # if previous was an operator, return false
382     beq $t0, 2, unbalanced
383     # if previous was '(', return false
384     beq $t0, 3, unbalanced
385     # set prev = 2 (for operator)
386     li $t0, 2
387     j lInvalidLoop
388
389     end_loop:
390     # if previous was '(', return false
391     beq $t0, 3, unbalanced
392     # if previous was an operator, return false
393     beq $t0, 2, unbalanced
394     # return 1 (true)
395     li $v0, 1
396     j end_expression
397
398     unbalanced:
399     # return 0 (false)
400     li $v0, 0
401
402     end_expression:
403     lw $ra, 0($sp)
404     lw $t0, 4($sp)
405     lw $t1, 8($sp)
406     lw $t2, 12($sp)
407     addi $sp, $sp, 16
408     jr $ra
```

Balance

c

```
179 bool Balance(string s)
180 {
181     int cnt=0;
182     for(char c:s)
183     {
184         if(c=='(')
185             cnt++;
186         else if(c==')')
187             cnt--;
188         if(cnt<0)
189             return false;
190     }
191     if(cnt)
192         return false;
193     return true;
194 }
```

Mips

```
410 Balance:
411     addi $sp, $sp, -4
412     sw $ra, 0($sp)
413
414     # counter i
415     li $t0, 0
416     # Loop over the string
417     loop2:
418     # load from standardExpression
419     lb $t1, 0($a0)
420     # if(null-terminator) break
421     beqz $t1, end_loop2
422     # moving to the next character
423     addi $a0, $a0, 1
424     # increment counter if '('
425     beq $t1, '(', increase_cnt
426     # decrement counter if ')'
427     beq $t1, ')', decrease_cnt
428     # loop
429     j loop2
430
431     increase_cnt:
432     # increment counter
433     addi $t0, $t0, 1
434     j loop2
```

```
436 decrease_cnt:
437     # decrement counter
438     addi $t0, $t0, -1
439     # if(counter < 0) return 0
440     bltz $t0, unbalanced2
441     j loop2
442
443 unbalanced2:
444     # return 0
445     li $v0, 0
446     j end_balance2
447
448 end_loop2:
449     # if(counter != 0) return 0
450     bnez $t0, unbalanced2
451     # else return 1
452     li $v0, 1
453
454 end_balance2:
455     lw $ra, 0($sp)
456     addi $sp, $sp, 4
457     jr $ra
```

5. Main Function: -

```
1 .data
2 # Global variables
3 filteredExp: .space 1200 # filtered expression
4 standardExp: .space 1200 # Assuming res array
5 minusMul: .asciiiz "(0-1)*"
6 minusDiv: .asciiiz "(0-1)/"
7 fail_msg: .asciiiz "Expression is not valid.\n"
8 restart: .asciiiz "please press any key in the keyboard to restart the program..."
9 restart_done: .asciiiz "Restart Done\n"
10 head: .word 0 # Pointer to the head of the linked list (initialized to null) # integer stack
11 stack_size: .word 0 # Size of the stack (initialized to 0) #integer stack
12 array_size: .word 0
13 string_array: .space 1000
14 tempStringSize: .word 0
15 newline: .asciiiz "\n"
16 space: .asciiiz " "
17 tempString: .space 36
18 expression: .space 1200
19 tempExpression: .space 1200
20 emptyStack: .asciiiz "Stack is empty\n"
21 prompt: .asciiiz "Enter infixExpression : "
22 valuPrint: .asciiiz "Value: "
23 POS_Str: .asciiiz "postfix Expression = "
24 STANDARD_STR: .asciiiz "Standard Expression = "
```

```
31
32 main:
33     la $a0, Promote
34     jal printString
35
36     la $a0, tempexpression
37     li $a1, 100
38     li $v0, 8
39     syscall
40
41     la $t0, tempexpression
42     la $t1, s
43     li $t3, ' '
44     li $t4, '\n'
45     Filterloop:
46         lb $t2, 0($t0)
47         beq $t2, $zero, null
48         beq $t2, $t3, incrementFilterLoop
49         beq $t2, $t4, incrementFilterLoop
50         sb $t2, 0($t1)
51         addi $t1, $t1, 1
52         incrementFilterLoop:
53             addi $t0, $t0, 1
54             j Filterloop
55
56     null:
57         sb $t2, 0($t1)
58         j exiteFilterLoop
59
60     exiteFilterLoop:
61
```

```
60     exiteFilterLoop:
61
62     jal to_standard_infinix
63     la $a0, STANDARD_STR # Load address of res
64     jal printString
65
66     la $a0, res # Load address of res
67     jal printString
68
69     jal printNewLine
70
71     # go to validation
72
73     # Load address of the string into $a0
74     la $a0, res
75
76     # Call the Bal function to check parentheses balance
77     jal Bal
78     beq $v0, $0, not_valid
79     # Call the isValidExpression function
80     la $a0, res
81     jal isValidExpression
82
83     # Check the result
84     bne $v0, $zero, valid
85
```

Mars Messages Run I/O

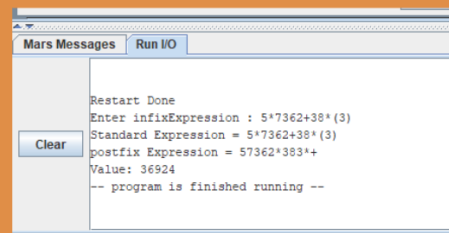
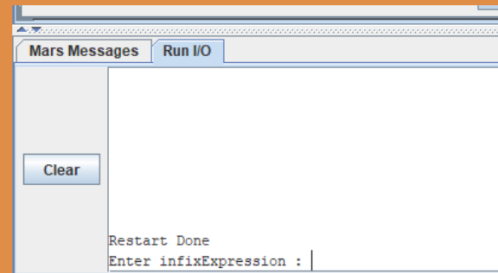
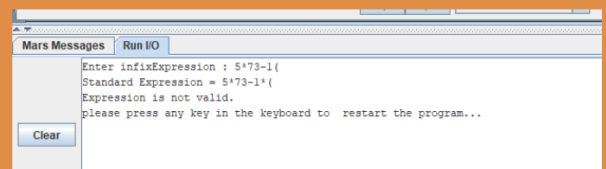
Enter infixExpression : 12*(23+83)-323*4723-112/-12

Clear


```

82
83     # Check the result
84     bne $v0, $zero, valid
85 not_valid:
86     la $a0, fail_msg
87     jal printString
88
89     la $a0, restart
90     jal printString
91
92     li $v0, 12 # system call for read char
93     syscall
94     li $t0, 0
95     li $t1, 20
96
97 newLineLOOP:
98     beq $t0, $t1, exit_newLineLOOP
99
100    jal printNewLine
101
102    addi $t0, $t0, 1
103    j newLineLOOP
104
105 exit_newLineLOOP:
106
107
108    la $a0, restart_done
109    jal printString
110
111    j main

```



```

112
113 valid:
114
115     la $t0, res
116     la $t1, expression
117
118 copyloop1:
119     lb $t2, 0($t0)
120     beq $t2, $zero, null12
121
122     sb $t2, 0($t1)
123     addi $t1, $t1, 1
124
125     addi $t0, $t0, 1
126     j copyloop1
127 null12:
128     sb $t2, 0($t1)
129     j exitcopyloop1
130
131 exitcopyloop1:
132
133 jal Infix_to_postfix
134
135     la $a0, POS_Str
136     jal printString
137
138     la $s4, string_array
139     li $s5, 0
140     lw $t6, array_size
141
142 print_postfix_loop:
143     bge $s5, $t6, end_print_postfix_loop
144     sll $t7, $s5, 2
145     add $t7, $t7, $s4
146     lw $t8, 0($t7)
147
148     move $a0, $t8
149     jal printString
150
151     addi $s5, $s5, 1
152     j print_postfix_loop
153 end_print_postfix_loop:

```

```

153     end_print_postfix_loop:
154         jal printNewLine
155
156         la $a0, string_array
157         jal Evaluation
158
159         # move result in $t0
160         move $t0, $v0
161         la $a0, valuPrint
162         jal printString
163
164         li $v0, 1
165         move $a0, $t0
166         syscall
167
168         li $v0, 10 # end program
169         syscall
170 #end of main function

```

Test Cases

Sample [Input/Output]:

Mars Messages Run I/O

Enter infixExpression : $-3^4-(196/13-(3+4)^2)*3$

Standard Expression = $(0-1)*3^4-(196/13-(3+4)^2)*3$

postfix Expression = $01-34^*19613/34+2^-3*-$

Value: 21

-- program is finished running --

Clear

System Input

Form STD. Expr.

Postfix Form

Value Output



Register Values:

Registers	Coproc 1	Coproc 0
Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x10010000
\$v0	2	0x0000000a
\$v1	3	0x00000000
\$a0	4	0x00000015
\$a1	5	0xffffffff9a
\$a2	6	0x0000002d
\$a3	7	0x00000000
\$t0	8	0x00000015
\$t1	9	0x10010dd5
\$t2	10	0x00000000
\$t3	11	0x00000013
\$t4	12	0x0000001b
\$t5	13	0x00000000
\$t6	14	0x00000013
\$t7	15	0x00000013
\$s0	16	0x00000001
\$s1	17	0x0000001c
\$s2	18	0x10010a30
\$s3	19	0xffffffff9a
\$s4	20	0xffffffffaf
\$s5	21	0x00000013
\$s6	22	0x00000000
\$s7	23	0x00000000
\$t8	24	0x10040138
\$t9	25	0x10040138
\$k0	26	0x00000000
\$k1	27	0x00000000
\$gp	28	0x10008000
\$sp	29	0x7ffffefc
\$fp	30	0x00000000
\$ra	31	0x00400180
pc		0x00400194
hi		0xffffffff
lo		0xffffffff9a



Memory Segment Values:

Text Segment

Bkpt	Address	Code	Basic	Source
	0x00400000	0x3c011001	lui \$1,0x00001001	29: la \$a0 , prompt
	0x00400004	0x34241768	ori \$4,\$1,0x00001768	
	0x00400008	0x0c1002f4	jal 0x00400bd0	30: jal printString # prompt message
	0x0040000c	0x3c011001	lui \$1,0x00001001	32: la \$a0 , tempExpression
	0x00400010	0x342412a8	ori \$4,\$1,0x000012a8	
	0x00400014	0x240500e4	addiu \$5,\$0,0x000000e4	33: li \$a1 , 100
	0x00400018	0x24020008	addiu \$2,\$0,0x00000008	34: li \$v0 , 8
	0x0040001c	0x0000000c	syscall	35: syscall # input expression
	0x00400020	0x3c011001	lui \$1,0x00001001	37: la \$t0 , tempExpression # expression's address
	0x00400024	0x342812a8	ori \$8,\$1,0x000012a8	
	0x00400028	0x3c011001	lui \$1,0x00001001	38: la \$t1 , filteredExp # filtered expression's address
	0x0040002c	0x34290000	ori \$9,\$1,0x00000000	
	0x00400030	0x240b0020	addiu \$11,\$0,0x0000...	39: li \$t3 , ' '
	0x00400034	0x240c000a	addiu \$12,\$0,0x0000...	40: li \$t4 , '\n'
	0x00400038	0x810a0000	lb \$t0,0x00000000(\$8)	42: lb \$t2 , 0(\$t0) # load from expression
	0x0040003c	0x11400006	beq \$t0,\$0,0x00000006	43: beq \$t2 , \$zero , null
	0x00400040	0x114b0003	beq \$t0,\$t1,0x00000003	44: beq \$t2 , \$t3 , incrementFilterLoop

Labels

Label	Address
(global)	
main	0x00400000
Final.asm	
FilterLoop	0x00400038
incrementFilterLoop	0x00400050
null	0x00400058
exiteFilterLoop	0x00400060
not_valid	0x004000a0
newLineLOOP	0x004000c8
exite_newLineLOOP	0x004000d8
valid	0x004000e8
copyloop1	0x004000f8
null12	0x00400110
exitecopyloop1	0x00400118
print_postfix_loop	0x0040013c
end_postfix_loop	0x00400150

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x345e332d	0x3931282d	0x33312f36	0x2b33282d	0x325e2934	0x00332a29	0x00000000	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010160	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010180	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

0x10010000 (.data)

☒ Hexadecimal Addresses
 ☒ Hexadecimal Values
 ☐ ASCII



Extra Examples:

```

Enter InfixExpression : 2^3-4+4^2+4*-3
Standard Expression  : 2^3-4+4^2+4*(0-1)*3
postfix Expression   : 23^4-42^+401-*3**
Value                : 8

-----

Enter infixExpression : 5*(12+8)-3*(4-2)
Standard Expression   : 5*(12+8)-3*(4-2)
postfix Expression    : 5128+*342-*
Value                 : 94

-----

Enter InfixExpression : -23*837*(-12/-(2*-5))+9-89)*3/12*72+(-1*23-+2)*4(4--2)(4++2)
Standard Expression   : (0-1)*23*837*((0-1)*12/(0-1)/(2*(0-1)*5)+9-89)*3/12*72+((0-1)*1*23-2)*4*(4-(0-1)*2)*(4+2)
postfix Expression    : 01-23*837*01-12*01-/201-*5*/9+89-*3*12/72*01-1*23*2-4*401-2*-*42+**
Value                 : 28064304

-----

Enter InfixExpression : -5*1234*(-8/-((3*-6)+7-56)*4/7*89+(-2*45-+8)*3*(6--3)*(6++3)
Standard Expression   : (0-1)*5*1234*((0-1)*8/(0-1)/(3*(0-1)*6)+7-56)*4/7*89+((0-1)*2*45-8)*3*(6-(0-1)*3)*(6+3)
postfix Expression    : 01-5*1234*01-8*01-/301-*6*/7+56-*4*7/89*01-2*45*8-3*601-3-*63+**
Value                 : 15351826

-----

Enter InfixExpression : 2^4-3*2*(3+3)/-4(4+5)(5--4)+(-4)^2
Standard Expression   : 2^4-3*2*(3+3)/(0-1)/4*(4+5)*(5-(0-1)*4)+((0-1)*4)^2
postfix Expression    : 24^32*33+*01-/4/45+*501-4*-*-01-4*2^+
Value                 : 761
  
```

Task Assignments

Each member participated in the project with: -

	Searching and writing code	Reviewing code	Code compilation	Report	presentation
Abdelrahman Mahmoud Mohamed	16.67%	16.67%	16.67%	16.67%	16.67%
Ahmed Bahy Youssef	16.67%	16.67%	16.67%	16.67%	16.67%
Ahmed Hossam Mohamed Fekry	16.67%	16.67%	16.67%	16.67%	16.67%
Ahmed Ragab Mahmoud	16.67%	16.67%	16.67%	16.67%	16.67%
Ahmed Sobhy Refaay	16.67%	16.67%	16.67%	16.67%	16.67%
Hassan Hussien Azmy	16.67%	16.67%	16.67%	16.67%	16.67%